Sandboxing AI agents, 100x faster

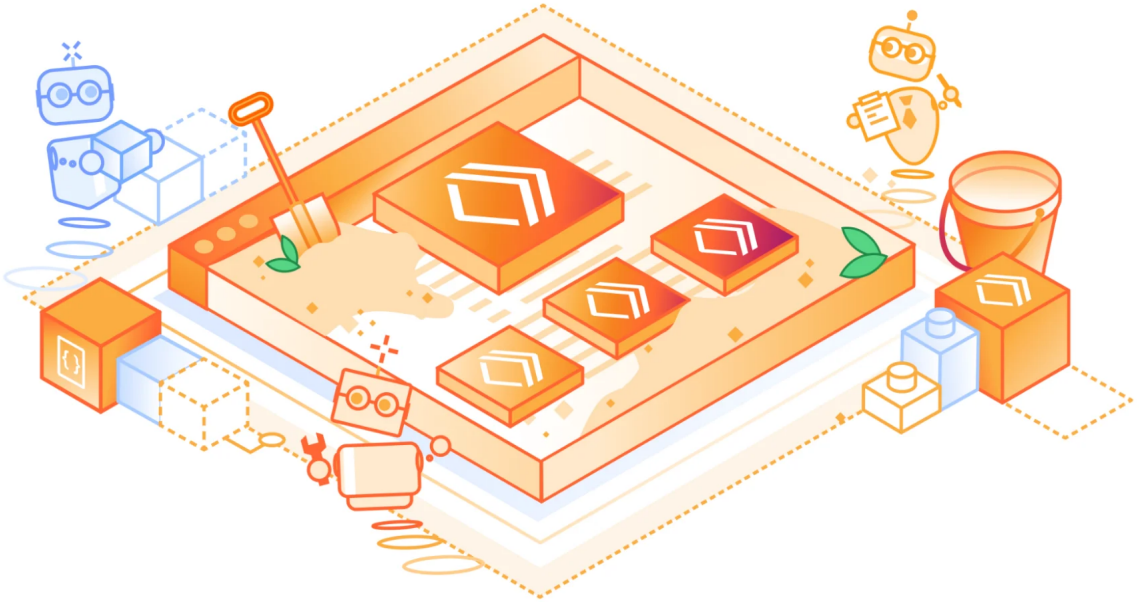
2026-03-24

Kenton Varda

Sunil Pai

Ketan Gupta

9 min read



Last September we introduced Code Mode, the idea that agents should perform tasks not by making tool calls, but instead by writing code that calls APIs. We've shown that simply converting an MCP server into a TypeScript API can cut token usage by 81%. We demonstrated that Code Mode can also operate *behind* an MCP server instead of in front of it, creating the new Cloudflare MCP server that exposes the entire Cloudflare API with just two tools and under 1,000 tokens.

But if an agent (or an MCP server) is going to execute code generated on-the-fly by AI to perform tasks, that code needs to run somewhere, and that somewhere needs to be secure. You can't just `eval()` AI-generated code directly in your app: a malicious user could trivially prompt the AI to inject vulnerabilities.

You need a **sandbox**: a place to execute code that is isolated from your application and from the rest of the world, except for the specific capabilities the code is meant to access.

Sandboxing is a hot topic in the AI industry. For this task, most people are reaching for containers. Using a Linux-based container, you can start up any sort of code execution environment you want. Cloudflare even offers our container runtime and our Sandbox SDK for this purpose.

But containers are expensive and slow to start, taking hundreds of milliseconds to boot and hundreds of megabytes of memory to run. You probably need to keep them warm to avoid delays, and you may be tempted to reuse existing containers for multiple tasks, compromising the security.

If we want to support consumer-scale agents, where every end user has an agent (or many!) and every agent writes code, containers are not enough. We need something lighter.

And we have it.

Dynamic Worker Loader: a lean sandbox

Tucked into our Code Mode post in September was the announcement of a new, experimental feature: the Dynamic Worker Loader API. This API allows a Cloudflare Worker to instantiate a new Worker, in its own sandbox, with code specified at runtime, all on the fly.

Dynamic Worker Loader is now in open beta, available to all paid Workers

users.

Read the docs for full details, but here's what it looks like:

```
// Have your LLM generate code like this.
let agentCode: string = `
  export default {
    async myAgent(param, env, ctx) {
      // ...
    }
  }
`;

// Get RPC stubs representing APIs the agent should be able
// to access. (This can be any Workers RPC API you define.)
let chatRoomRpcStub = ...;

// Load a worker to run the code, using the worker loader
// binding.
let worker = env.LOADER.load({
  // Specify the code.
  compatibilityDate: "2026-03-01",
  mainModule: "agent.js",
  modules: { "agent.js": agentCode },

  // Give agent access to the chat room API.
  env: { CHAT_ROOM: chatRoomRpcStub },

  // Block internet access. (You can also intercept it.)
  globalOutbound: null,
});

// Call RPC methods exported by the agent code.
await worker.getEntrypoint().myAgent(param);
```

That's it.

100x faster

Dynamic Workers use the same underlying sandboxing mechanism that the entire Cloudflare Workers platform has been built on since its launch, eight years ago: isolates. An isolate is an instance of the V8 JavaScript execution engine, the same engine used by Google Chrome. They are how Workers work.

An isolate takes a few milliseconds to start and uses a few megabytes of memory. That's around 100x faster and 10x-100x more memory efficient than a typical container.

That means that if you want to start a new isolate for every user request, on-demand, to run one snippet of code, then throw it away, you can.

Unlimited scalability

Many container-based sandbox providers impose limits on global concurrent sandboxes and rate of sandbox creation. Dynamic Worker Loader has no such limits. It doesn't need to, because it is simply an API to the same technology that has powered our platform all along, which has always allowed Workers to seamlessly scale to millions of requests per second.

Want to handle a million requests per second, where *every single request* loads a separate Dynamic Worker sandbox, all running concurrently? No problem!

Zero latency

One-off Dynamic Workers usually run on the same machine — the same thread, even — as the Worker that created them. No need to communicate around the world to find a warm sandbox. Isolates are so lightweight that we can just run them wherever the request landed. Dynamic Workers are supported in every one of Cloudflare's hundreds of locations around the world.

It's all JavaScript

The only catch, vs. containers, is that your agent needs to write JavaScript.

Technically, Workers (including dynamic ones) can use Python and WebAssembly, but for small snippets of code — like that written on-demand by an agent — JavaScript will load and run much faster.

We humans tend to have strong preferences on programming languages, and while many love JavaScript, others might prefer Python, Rust, or countless others.

But we aren't talking about humans here. We're talking about AI. AI will write any language you want it to. LLMs are experts in every major language. Their training data in JavaScript is immense.

JavaScript, by its nature on the web, is designed to be sandboxed. It is the correct language for the job.

Tools defined in TypeScript

If we want our agent to be able to do anything useful, it needs to talk to external APIs. How do we tell it about the APIs it has access to?

MCP defines schemas for flat tool calls, but not programming APIs. OpenAPI offers a way to express REST APIs, but it is verbose, both in the schema itself and the code you'd have to write to call it.

For APIs exposed to JavaScript, there is a single, obvious answer: TypeScript.

Agents know TypeScript. TypeScript is designed to be concise. With very few tokens, you can give your agent a precise understanding of your API.

```
// Interface to interact with a chat room.
interface ChatRoom {
```

```
  // Get the last `limit` messages of the chat log.
  getHistory(limit: number): Promise<Message[]>;

  // Subscribe to new messages. Dispose the returned object
  // to unsubscribe.
  subscribe(callback: (msg: Message) => void): Promise<Disposable>;

  // Post a message to chat.
  post(text: string): Promise<void>;
}

type Message = {
  author: string;
  time: Date;
  text: string;
}
```

Compare this with the equivalent OpenAPI spec (which is so long you have to scroll to see it all):

```
openapi: 3.1.0
info:
  title: ChatRoom API
  description: >
    Interface to interact with a chat room.
  version: 1.0.0

paths:
  /messages:
    get:
      operationId: getHistory
      summary: Get recent chat history
      description: Returns the last `limit` messages from the chat
log, newest first.
      parameters:
```

We think the TypeScript API is better. It's fewer tokens and much easier to understand (for both agents and humans).

Dynamic Worker Loader makes it easy to implement a TypeScript API like this in your own Worker and then pass it in to the Dynamic Worker either as a method parameter or in the env object. The Workers Runtime will automatically set up a Cap'n Web RPC bridge between the sandbox and your harness code, so that the agent can invoke your API across the security boundary without ever realizing that it isn't using a local library.

That means your agent can write code like this:

```
// Thinking: The user asked me to summarize recent chat messages from
Alice.
// I will filter the recent message history in code so that I only have
to
// read the relevant messages.
let history = await env.CHAT_ROOM.getHistory(1000);
return history.filter(msg => msg.author == "alice");
```

HTTP filtering and credential injection

If you prefer to give your agents HTTP APIs, that's fully supported. Using the `globalOutbound` option to the worker loader API, you can register a callback to be invoked on every HTTP request, in which you can inspect the request, rewrite it, inject auth keys, respond to it directly, block it, or anything else you might like.

For example, you can use this to implement **credential injection** (token injection): When the agent makes an HTTP request to a service that requires authorization, you add credentials to the request on the way out. This way, the agent itself never knows the secret credentials, and therefore cannot leak them.

Using a plain HTTP interface may be desirable when an agent is talking to a well-known API that is in its training set, or when you want your agent to use a library that is built on a REST API (the library can run inside the

agent's sandbox).

With that said, **in the absence of a compatibility requirement, TypeScript RPC interfaces are better than HTTP:**

- As shown above, a TypeScript interface requires far fewer tokens to describe than an HTTP interface.
- The agent can write code to call TypeScript interfaces using far fewer tokens than equivalent HTTP.
- With TypeScript interfaces, since you are defining your own wrapper interface anyway, it is easier to narrow the interface to expose exactly the capabilities that you want to provide to your agent, both for simplicity and security. With HTTP, you are more likely implementing *filtering* of requests made against some existing API. This is hard, because your proxy must fully interpret the meaning of every API call in order to properly decide whether to allow it, and HTTP requests are complicated, with many headers and other parameters that could all be meaningful. It ends up being easier to just write a TypeScript wrapper that only implements the functions you want to allow.

Battle-hardened security

Hardening an isolate-based sandbox is tricky, as it is a more complicated attack surface than hardware virtual machines. Although all sandboxing mechanisms have bugs, security bugs in V8 are more common than security bugs in typical hypervisors. When using isolates to sandbox possibly-malicious code, it's important to have additional layers of defense-in-depth. Google Chrome, for example, implemented strict process isolation for this reason, but it is not the only possible solution.

We have nearly a decade of experience securing our isolate-based

platform. Our systems automatically deploy V8 security patches to production within hours — faster than Chrome itself. Our security architecture features a custom second-layer sandbox with dynamic cordoning of tenants based on risk assessments. We've extended the V8 sandbox itself to leverage hardware features like MPK. We've teamed up with (and hired) leading researchers to develop novel defenses against Spectre. We also have systems that scan code for malicious patterns and automatically block them or apply additional layers of sandboxing. And much more.

When you use Dynamic Workers on Cloudflare, you get all of this automatically.

Helper libraries

We've built a number of libraries that you might find useful when working with Dynamic Workers:

Code Mode

`@cloudflare/codemode` simplifies running model-generated code against AI tools using Dynamic Workers. At its core is `DynamicWorkerExecutor()`, which constructs a purpose-built sandbox with code normalisation to handle common formatting errors, and direct access to a `globalOutbound` fetcher for controlling `fetch()` behaviour inside the sandbox — set it to `null` for full isolation, or pass a `Fetcher` binding to route, intercept or enrich outbound requests from the sandbox.

```
const executor = new DynamicWorkerExecutor({
  loader: env.LOADER,
  globalOutbound: null, // fully isolated
});

const codemode = createCodeTool({
  tools: myTools,
```

```
    executor,
  });

  return generateText({
    model,
    messages,
    tools: { codemode },
  });
```

The Code Mode SDK also provides two server-side utility functions. `codeMcpServer({ server, executor })` wraps an existing MCP Server, replacing its tool surface with a single `code()` tool. `openApiMcpServer({ spec, executor, request })` goes further: given an OpenAPI spec and an executor, it builds a complete MCP Server with `search()` and `execute()` tools as used by the Cloudflare MCP Server, and better suited to larger APIs.

In both cases, the code generated by the model runs inside Dynamic Workers, with calls to external services made over RPC bindings passed to the executor.

Learn more about the library and how to use it.

Bundling

Dynamic Workers expect pre-bundled modules. `@cloudflare/worker-bundler` handles that for you: give it source files and a `package.json`, and it resolves npm dependencies from the registry, bundles everything with `esbuild`, and returns the module map the Worker Loader expects.

```
import { createWorker } from "@cloudflare/worker-bundler";

const worker = env.LOADER.get("my-worker", async () => {
  const { mainModule, modules } = await createWorker({
    files: {
      "src/index.ts": `
```

```
        import { Hono } from 'hono';
        import { cors } from 'hono/cors';

        const app = new Hono();
        app.use('*', cors());
        app.get('/', (c) => c.text('Hello from Hono!'));
        app.get('/json', (c) => c.json({ message: 'It works!' }));

        export default app;
      `,
      "package.json": JSON.stringify({
        dependencies: { hono: "^4.0.0" }
      })
    }
  });

  return { mainModule, modules, compatibilityDate: "2026-01-01" };
});

await worker.getEntrypoint().fetch(request);
```

It also supports full-stack apps via `createApp` — bundle a server Worker, client-side JavaScript, and static assets together, with built-in asset serving that handles content types, ETags, and SPA routing.

Learn more about the library and how to use it.

File manipulation

`@cloudflare/shell` gives your agent a virtual filesystem inside a Dynamic Worker. Agent code calls typed methods on a `state` object — read, write, search, replace, diff, glob, JSON query/update, archive — with structured inputs and outputs instead of string parsing.

Storage is backed by a durable `Workspace` (SQLite + R2), so files persist across executions. Coarse operations like `searchFiles`, `replaceInFiles`, and `planEdits` minimize RPC round-trips — the agent issues one call

instead of looping over individual files. Batch writes are transactional by default: if any write fails, earlier writes roll back automatically.

```
import { Workspace } from "@cloudflare/shell";
import { stateTools } from "@cloudflare/shell/workers";
import { DynamicWorkerExecutor, resolveProvider } from "@cloudflare/
codemode";

const workspace = new Workspace({
  sql: this.ctx.storage.sql, // Works with any DO's SqlStorage, D1, or
custom SQL backend
  r2: this.env.MY_BUCKET, // large files spill to R2 automatically
  name: () => this.name   // lazy — resolved when needed, not at
construction
});

// Code runs in an isolated Worker sandbox with no network access
const executor = new DynamicWorkerExecutor({ loader: env.LOADER });

// The LLM writes this code; `state.*` calls dispatch back to the host
via RPC
const result = await executor.execute(
  `async () => {
    // Search across all TypeScript files for a pattern
    const hits = await state.searchFiles("src/**/*.ts", "answer");
    // Plan multiple edits as a single transaction
    const plan = await state.planEdits([
      { kind: "replace", path: "/src/app.ts",
        search: "42", replacement: "43" },
      { kind: "writeJson", path: "/src/config.json",
        value: { version: 2 } }
    ]);
    // Apply atomically — rolls back on failure
    return await state.applyEditPlan(plan);
  }`,
  [resolveProvider(stateTools(workspace))]
);
```

The package also ships prebuilt TypeScript type declarations and a system prompt template, so you can drop the full `state` API into your LLM context in a handful of tokens.

Learn more about the library and how to use it.

How are people using it?

Code Mode

Developers want their agents to write and execute code against tool APIs, rather than making sequential tool calls one at a time. With Dynamic Workers, the LLM generates a single TypeScript function that chains multiple API calls together, runs it in a Dynamic Worker, and returns the final result back to the agent. As a result, only the output, and not every intermediate step, ends up in the context window. This cuts both latency and token usage, and produces better results, especially when the tool surface is large.

Our own Cloudflare MCP server is built exactly this way: it exposes the entire Cloudflare API through just two tools — search and execute — in under 1,000 tokens, because the agent writes code against a typed API instead of navigating hundreds of individual tool definitions.

Building custom automations

Developers are using Dynamic Workers to let agents build custom automations on the fly. Zite, for example, is building an app platform where users interact through a chat interface — the LLM writes TypeScript behind the scenes to build CRUD apps, connect to services like Stripe, Airtable, and Google Calendar, and run backend logic, all without the user ever seeing a line of code. Every automation runs in its own Dynamic Worker, with access to only the specific services and libraries that the endpoint needs.

> *"To enable server-side code for Zite's LLM-generated apps, we needed an execution layer that was instant, isolated, and secure. Cloudflare's Dynamic Workers hit the mark on all three, and out-performed all of the other platforms we benchmarked for speed and library support. The NodeJS compatible runtime supported all of Zite's workflows, allowing hundreds of third party integrations, without sacrificing on startup time. Zite now services millions of execution requests daily thanks to Dynamic Workers."*
>
> *— Antony Toron, CTO and Co-Founder, Zite*

Running AI-generated applications

Developers are building platforms that generate full applications from AI — either for their customers or for internal teams building prototypes. With Dynamic Workers, each app can be spun up on demand, then put back into cold storage until it's invoked again. Fast startup times make it easy to preview changes during active development. Platforms can also block or intercept any network requests the generated code makes, keeping AI-generated apps safe to run.

Pricing

Dynamically-loaded Workers are priced at $0.002 per unique Worker loaded per day (as of this post's publication), in addition to the usual CPU time and invocation pricing of regular Workers.

For AI-generated "code mode" use cases, where every Worker is a unique one-off, this means the price is $0.002 per Worker loaded (plus CPU and invocations). This cost is typically negligible compared to the inference costs to generate the code.

During the beta period, the $0.002 charge is waived. As pricing is subject to change, please always check our Dynamic Workers pricing for the most

current information.

Get Started

If you're on the Workers Paid plan, you can start using Dynamic Workers today.

Dynamic Workers Starter

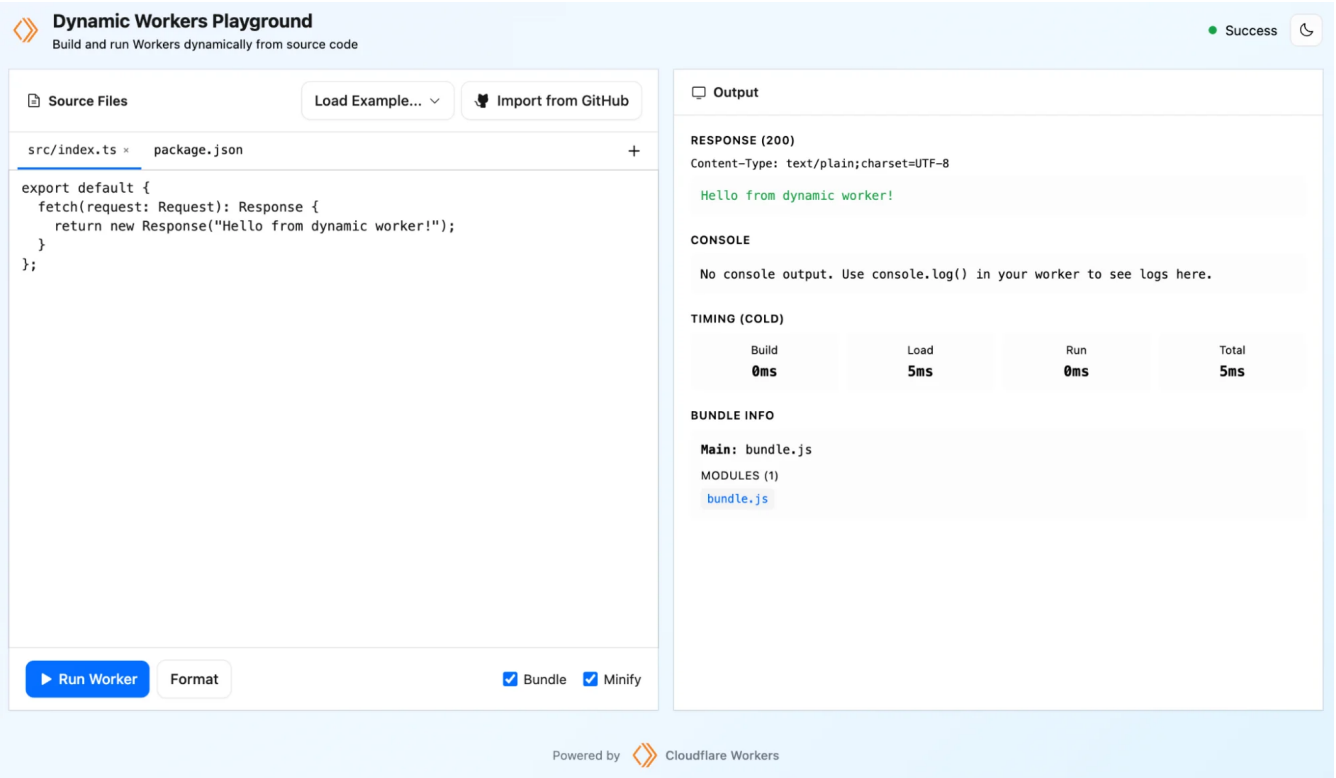
Deploy to Cloudflare

Use this "hello world" starter to get a Worker deployed that can load and execute Dynamic Workers.

Dynamic Workers Playground

Deploy to Cloudflare

You can also deploy the Dynamic Workers Playground, where you'll be able to write or import code, bundle it at runtime with `@cloudflare/worker-bundler`, execute it through a Dynamic Worker, see real-time responses and execution logs.



Dynamic Workers are fast, scalable, and lightweight. Find us on Discord if you have any questions. We'd love to see what you build!

Cloudflare's connectivity cloud protects entire corporate networks, helps customers build Internet-scale applications efficiently, accelerates any website or Internet application, wards off DDoS attacks, keeps hackers at bay, and can help you on your journey to Zero Trust.

Visit 1.1.1.1 from any device to get started with our free app that makes your Internet faster and safer.

To learn more about our mission to help build a better Internet, start here. If you're looking for a new career direction, check out our open positions.

Discuss on Hacker News

On Air

Developer Week Day 2: Fullstack on Cloudflare

Tune In

MCP Workers AI AI Agents Developer Platform Developers

Follow on X

Kenton Varda | @kentonvarda
Sunil Pai | @threepointone
Cloudflare | @cloudflare

RELATED POSTS

March 19, 2026 8:53 PM

Powering the agents: Workers AI now runs large models, starting with Kimi K2.5

Kimi K2.5 is now on Workers AI, helping you power agents entirely on Cloudflare's Developer Platform. Learn how we optimized our inference stack and reduced inference costs for internal agent use cases. ...

By Michelle Chen, Kevin Flansburg, Ashish Datta, Kevin Jain

Developer Platform, Developers, Workers AI, Cloudflare Workers, AI, Agents

March 11, 2026 2:05 PM

Slashing agent token costs by 98% with RFC 9457-compliant error responses

Cloudflare now returns RFC 9457-compliant structured Markdown and JSON error payloads to AI agents, replacing heavyweight HTML pages with machine-readable instructions. This reduces token usage by over 98%, turning brittle parsing into efficient control flow....

By Sam Marsh

AI, Developer Platform, Developers, WAF, Edge Computing

March 11, 2026 2:00 PM

AI Security for Apps is now generally available

Cloudflare AI Security for Apps is now generally available, providing a security layer to discover and protect AI-powered applications, regardless of the model or hosting

provider. We are also making AI discovery free for all plans, to help teams find and secure shadow AI deployments....

By Liam Reese, Zhiyuan Zheng, Catherine Newcomb

Product News, AI, WAF, Security, Application Security, Application Services

March 11, 2026 2:00 PM

AI Security for Apps is now generally available

Cloudflare AI Security for Apps is now generally available, providing a security layer to discover and protect AI-powered applications, regardless of the model or hosting provider. We are also making AI discovery free for all plans, to help teams find and secure shadow AI deployments....

By Liam Reese, Zhiyuan Zheng, Catherine Newcomb

Product News, AI, WAF, Security, Application Security, Application Services

© 2026 Cloudflare, Inc. | Privacy Policy | Terms of Use | Report Security Issues | Cookie Preferences| Trademark